CODE	OUTPUT

```
 #include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node
{
   char id[20];
   struct Node *left;
   struct Node *right;
};
struct Node* createNode(char id[])
{
   struct Node *newNode;

   newNode = (struct
Node*)malloc(sizeof(struct Node));

   strcpy(newNode->id, id);
   newNode->left = NULL;
   newNode->right = NULL;

   return newNode;
}
struct Node* insert(struct Node *root, char
id[], int *comparisons)
{
   if (root == NULL)
      return createNode(id);

   (*comparisons)++;

   if (strcmp(id, root->id) < 0)
      root->left = insert(root->left, id,
comparisons);
   else if (strcmp(id, root->id) > 0)
      root->right = insert(root->right, id,
comparisons);

   return root;
}
void inorder(struct Node *root)
{
   if (root != NULL)
   {
      inorder(root->left);
      printf("%s ", root->id);
      inorder(root->right);
   }
```

```
GOVERNMENT DATABASE - BST
ASSIGNMENT
-----------------------------------

Insertion Trace:
ID          Comparisons
A102         0
A25          1
A7           2
B100         3
B12          4
A120         2
B3           5
A45          3

Inorder Traversal:
A102 A120 A25 A45 A7 B100 B12 B3

BST Height = 5 edges
Number of Levels = 6


SEARCH COMPARISON
-----------------------------------
ID          BST    Linear
A102         1     1
A45          4     8
B3           6     7
A120         3     6
B12          5     5
A7           3     3

COMPLEXITY ANALYSIS
-----------------------------------
BST Search - Best Case: O(1)
BST Search - Average Case: O(log n)
BST Search - Worst Case: O(n)
Linear Search - Best Case: O(1)
Linear Search - Average Case: O(n)
Linear Search - Worst Case: O(n)
```

```
}
int bstSearch(struct Node *root, char key[],
int *comparisons)
{
   struct Node *current = root;

   while (current != NULL)
   {
      (*comparisons)++;

      if (strcmp(key, current->id) == 0)
         return 1;

      if (strcmp(key, current->id) < 0)
         current = current->left;
      else
         current = current->right;
   }
  return 0;
}
int linearSearch(char ids[][20], int n, char
key[], int *comparisons)
{
   int i;

   for (i = 0; i < n; i++)
   {
      (*comparisons)++;

      if (strcmp(ids[i], key) == 0)
         return 1;
   }
   return 0;
int height(struct Node *root)
{
   int leftHeight, rightHeight;
    if (root == NULL)
      return -1;

   leftHeight = height(root->left);
   rightHeight = height(root->right);
   if (leftHeight > rightHeight)
      return leftHeight + 1;
   else
      return rightHeight + 1;
}
void freeTree(struct Node *root)
{
   if (root != NULL)
   {
      freeTree(root->left);
```

```
    freeTree(root->right);
    free(root);
  }
}
int main()
{
  char ids[8][20] =
  {
    "A102",
    "A25",
    "A7",
    "B100",
    "B12",
    "A120",
    "B3",
    "A45"
  };
  char searchKeys[6][20] =
  {
    "A102",
    "A45",
    "B3",
    "A120",
    "B12",
    "A7"
  };

  struct Node *root = NULL;

  int i;
  int comparisons;
  int bstComp, linearComp;
  int n = 8;

  printf("GOVERNMENT DATABASE - BST
ASSIGNMENT\n");
  printf("-----------------------------------\n\n");

  /* Part A - BST insertion */
  printf("Insertion Trace:\n");
  printf("ID\t\tComparisons\n");

  for (i = 0; i < n; i++)
  {
    comparisons = 0;
    root = insert(root, ids[i], &comparisons);
    printf("%s\t\t%d\n", ids[i], comparisons);
  }
  printf("\nInorder Traversal:\n");
  inorder(root);
```

```
   printf("\n\nBST Height = %d edges\n",
height(root));
   printf("Number of Levels = %d\n",
height(root) + 1);
   printf("\n\nSEARCH COMPARISON\n");
   printf("-----------------------------------\n");
   printf("ID\t\tBST\tLinear\n");

   for (i = 0; i < 6; i++)
   {
      bstComp = 0;
      linearComp = 0;

      bstSearch(root, searchKeys[i],
&bstComp);
      linearSearch(ids, n, searchKeys[i],
&linearComp);

      printf("%s\t\t%d\t%d\n",
           searchKeys[i], bstComp, linearComp);
   }
   printf("\nCOMPLEXITY ANALYSIS\n");
   printf("-----------------------------------\n");
   printf("BST Search - Best Case: O(1)\n");
   printf("BST Search - Average Case: O(log
n)\n");
   printf("BST Search - Worst Case: O(n)\n");
   printf("Linear Search - Best Case: O(1)\n");
   printf("Linear Search - Average Case:
O(n)\n");
   printf("Linear Search - Worst Case:
O(n)\n");
    freeTree(root);

      return 0;
}
```